

PocketSmart AI

Your Smart Budget & Recommendation Assistant

100-Page Detailed Project Report

Brainstorming & Ideation • Requirement Analysis • Project Design • Project Planning • Development •
Testing • Documentation • Demonstration

Academic Project Documentation

Project Roadmap and Page Allocation

- 1. Brainstorming and Ideation** — Pages 2–14
- 2. Requirement Analysis** — Pages 15–28
- 3. Project Design** — Pages 29–43
- 4. Project Planning** — Pages 44–55
- 5. Development** — Pages 56–77
- 6. Testing** — Pages 78–87
- 7. Documentation** — Pages 88–94
- 8. Demonstration** — Pages 95–100

This report is intentionally detailed and organized as an end-to-end academic project record. Each project phase explains objectives, activities, decisions, outputs, risks, controls, and expected evidence.

Brainstorming and Ideation — Page 3: Problem Statement

Page focus: Identify a practical personal-finance problem that can be solved with software. This page provides detailed project-level guidance for **problem statement** within PocketSmart AI.

1. Purpose: This page develops the problem statement aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats problem statement as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, problem statement must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the problem statement activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's problem statement record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the problem statement aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 4: Target Users

Page focus: Understand how users currently record income, expenses, and savings goals. This page provides detailed project-level guidance for **target users** within PocketSmart AI.

1. Purpose: This page develops the target users aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats target users as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, target users must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the target users activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's target users record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the target users aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 5: Pain Points

Page focus: Turn common budgeting difficulties into clearly defined product opportunities. This page provides detailed project-level guidance for **pain points** within PocketSmart AI.

1. Purpose: This page develops the pain points aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats pain points as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, pain points must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the pain points activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's pain points record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the pain points aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 6: User Personas

Page focus: Explore artificial intelligence as a support mechanism rather than a replacement for user control. This page provides detailed project-level guidance for **user personas** within PocketSmart AI.

1. Purpose: This page develops the user personas aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats user personas as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, user personas must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the user personas activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's user personas record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the user personas aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 7: Idea Generation

Page focus: Define a realistic project scope suitable for an academic software project. This page provides detailed project-level guidance for **idea generation** within PocketSmart AI.

1. Purpose: This page develops the idea generation aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats idea generation as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, idea generation must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the idea generation activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's idea generation record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the idea generation aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 8: Feature Prioritization

Page focus: Identify a practical personal-finance problem that can be solved with software. This page provides detailed project-level guidance for **feature prioritization** within PocketSmart AI.

1. Purpose: This page develops the feature prioritization aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats feature prioritization as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, feature prioritization must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the feature prioritization activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's feature prioritization record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the feature prioritization aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 9: Value Proposition

Page focus: Understand how users currently record income, expenses, and savings goals. This page provides detailed project-level guidance for **value proposition** within PocketSmart AI.

1. Purpose: This page develops the value proposition aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats value proposition as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, value proposition must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the value proposition activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's value proposition record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the value proposition aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 10: Ai Opportunities

Page focus: Turn common budgeting difficulties into clearly defined product opportunities. This page provides detailed project-level guidance for **AI opportunities** within PocketSmart AI.

1. Purpose: This page develops the AI opportunities aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats AI opportunities as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, AI opportunities must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the AI opportunities activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's AI opportunities record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the AI opportunities aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 11: Ethical Considerations

Page focus: Explore artificial intelligence as a support mechanism rather than a replacement for user control. This page provides detailed project-level guidance for **ethical considerations** within PocketSmart AI.

1. Purpose: This page develops the ethical considerations aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats ethical considerations as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, ethical considerations must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the ethical considerations activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's ethical considerations record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the ethical considerations aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 12: Competitive Observation

Page focus: Define a realistic project scope suitable for an academic software project. This page provides detailed project-level guidance for **competitive observation** within PocketSmart AI.

1. Purpose: This page develops the competitive observation aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats competitive observation as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, competitive observation must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the competitive observation activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's competitive observation record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.

34. Purpose: This page develops the competitive observation aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 13: Assumptions

Page focus: Identify a practical personal-finance problem that can be solved with software. This page provides detailed project-level guidance for **assumptions** within PocketSmart AI.

1. Purpose: This page develops the assumptions aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats assumptions as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, assumptions must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the assumptions activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's assumptions record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the assumptions aspect of PocketSmart AI in a structured, evidence-oriented manner.

Brainstorming and Ideation — Page 14: Constraints

Page focus: Understand how users currently record income, expenses, and savings goals. This page provides detailed project-level guidance for **constraints** within PocketSmart AI.

1. Purpose: This page develops the constraints aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats constraints as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, constraints must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the constraints activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's constraints record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the constraints aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 15: Functional Requirements

Page focus: Convert the selected idea into measurable functional and non-functional requirements. This page provides detailed project-level guidance for **functional requirements** within PocketSmart AI.

1. Purpose: This page develops the functional requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats functional requirements as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, functional requirements must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the functional requirements activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's functional requirements record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.

34. Purpose: This page develops the functional requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 16: Non-Functional Requirements

Page focus: Define what the system must do and how well it must perform. This page provides detailed project-level guidance for **non-functional requirements** within PocketSmart AI.

1. Purpose: This page develops the non-functional requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats non-functional requirements as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, non-functional requirements must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the non-functional requirements activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's non-functional requirements record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.

33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.

34. Purpose: This page develops the non-functional requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 17: User Stories

Page focus: Capture user expectations as use cases and user stories. This page provides detailed project-level guidance for **user stories** within PocketSmart AI.

1. Purpose: This page develops the user stories aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats user stories as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, user stories must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the user stories activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's user stories record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the user stories aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 18: Use Cases

Page focus: Identify data inputs, outputs, validation rules, and system constraints. This page provides detailed project-level guidance for **use cases** within PocketSmart AI.

1. Purpose: This page develops the use cases aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats use cases as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, use cases must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the use cases activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's use cases record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the use cases aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 19: Actors

Page focus: Create a traceable baseline for design, development, and testing. This page provides detailed project-level guidance for **actors** within PocketSmart AI.

1. Purpose: This page develops the actors aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats actors as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, actors must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the actors activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's actors record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the actors aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 20: Data Requirements

Page focus: Convert the selected idea into measurable functional and non-functional requirements. This page provides detailed project-level guidance for **data requirements** within PocketSmart AI.

1. Purpose: This page develops the data requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats data requirements as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, data requirements must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the data requirements activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's data requirements record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the data requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 21: Security Requirements

Page focus: Define what the system must do and how well it must perform. This page provides detailed project-level guidance for **security requirements** within PocketSmart AI.

1. Purpose: This page develops the security requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats security requirements as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, security requirements must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the security requirements activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's security requirements record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the security requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 22: Privacy Requirements

Page focus: Capture user expectations as use cases and user stories. This page provides detailed project-level guidance for **privacy requirements** within PocketSmart AI.

1. Purpose: This page develops the privacy requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats privacy requirements as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, privacy requirements must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the privacy requirements activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's privacy requirements record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the privacy requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 23: Performance Requirements

Page focus: Identify data inputs, outputs, validation rules, and system constraints. This page provides detailed project-level guidance for **performance requirements** within PocketSmart AI.

1. Purpose: This page develops the performance requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats performance requirements as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, performance requirements must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the performance requirements activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's performance requirements record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.

34. Purpose: This page develops the performance requirements aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 24: Availability

Page focus: Create a traceable baseline for design, development, and testing. This page provides detailed project-level guidance for **availability** within PocketSmart AI.

1. Purpose: This page develops the availability aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats availability as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, availability must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the availability activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's availability record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the availability aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 25: Usability

Page focus: Convert the selected idea into measurable functional and non-functional requirements. This page provides detailed project-level guidance for **usability** within PocketSmart AI.

1. Purpose: This page develops the usability aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats usability as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, usability must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the usability activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's usability record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the usability aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 26: Accessibility

Page focus: Define what the system must do and how well it must perform. This page provides detailed project-level guidance for **accessibility** within PocketSmart AI.

1. Purpose: This page develops the accessibility aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats accessibility as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, accessibility must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the accessibility activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's accessibility record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the accessibility aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 27: Validation

Page focus: Capture user expectations as use cases and user stories. This page provides detailed project-level guidance for **validation** within PocketSmart AI.

1. Purpose: This page develops the validation aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats validation as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, validation must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the validation activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's validation record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the validation aspect of PocketSmart AI in a structured, evidence-oriented manner.

Requirement Analysis — Page 28: Error Handling

Page focus: Identify data inputs, outputs, validation rules, and system constraints. This page provides detailed project-level guidance for **error handling** within PocketSmart AI.

1. Purpose: This page develops the error handling aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats error handling as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, error handling must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the error handling activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's error handling record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the error handling aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 29: Architecture

Page focus: Translate requirements into a maintainable software architecture. This page provides detailed project-level guidance for **architecture** within PocketSmart AI.

1. Purpose: This page develops the architecture aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats architecture as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, architecture must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the architecture activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's architecture record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the architecture aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 30: Module Design

Page focus: Separate user interface, application logic, data management, and recommendation services. This page provides detailed project-level guidance for **module design** within PocketSmart AI.

1. Purpose: This page develops the module design aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats module design as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, module design must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the module design activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's module design record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the module design aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 31: Database Design

Page focus: Design data structures that support reliable financial calculations. This page provides detailed project-level guidance for **database design** within PocketSmart AI.

1. Purpose: This page develops the database design aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats database design as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, database design must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the database design activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's database design record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the database design aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 32: Er Model

Page focus: Define safe interfaces between components. This page provides detailed project-level guidance for **ER model** within PocketSmart AI.

1. Purpose: This page develops the ER model aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats ER model as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, ER model must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the ER model activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's ER model record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the ER model aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 33: Api Design

Page focus: Prepare diagrams and design decisions that guide implementation. This page provides detailed project-level guidance for **API design** within PocketSmart AI.

1. Purpose: This page develops the API design aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats API design as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, API design must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the API design activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's API design record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the API design aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 34: Ui Design

Page focus: Translate requirements into a maintainable software architecture. This page provides detailed project-level guidance for **UI design** within PocketSmart AI.

1. Purpose: This page develops the UI design aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats UI design as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, UI design must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the UI design activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's UI design record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the UI design aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 35: Dashboard Design

Page focus: Separate user interface, application logic, data management, and recommendation services. This page provides detailed project-level guidance for **dashboard design** within PocketSmart AI.

1. Purpose: This page develops the dashboard design aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats dashboard design as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, dashboard design must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the dashboard design activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's dashboard design record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the dashboard design aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 36: Recommendation Engine

Page focus: Design data structures that support reliable financial calculations. This page provides detailed project-level guidance for **recommendation engine** within PocketSmart AI.

1. Purpose: This page develops the recommendation engine aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats recommendation engine as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, recommendation engine must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the recommendation engine activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's recommendation engine record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.

34. Purpose: This page develops the recommendation engine aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 37: Authentication

Page focus: Define safe interfaces between components. This page provides detailed project-level guidance for **authentication** within PocketSmart AI.

1. Purpose: This page develops the authentication aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats authentication as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, authentication must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the authentication activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's authentication record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the authentication aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 38: Authorization

Page focus: Prepare diagrams and design decisions that guide implementation. This page provides detailed project-level guidance for **authorization** within PocketSmart AI.

1. Purpose: This page develops the authorization aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats authorization as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, authorization must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the authorization activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's authorization record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the authorization aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 39: Data Validation

Page focus: Translate requirements into a maintainable software architecture. This page provides detailed project-level guidance for **data validation** within PocketSmart AI.

1. Purpose: This page develops the data validation aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats data validation as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, data validation must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the data validation activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's data validation record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the data validation aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 40: Logging

Page focus: Separate user interface, application logic, data management, and recommendation services. This page provides detailed project-level guidance for **logging** within PocketSmart AI.

1. Purpose: This page develops the logging aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats logging as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, logging must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the logging activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's logging record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the logging aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 41: Error Management

Page focus: Design data structures that support reliable financial calculations. This page provides detailed project-level guidance for **error management** within PocketSmart AI.

1. Purpose: This page develops the error management aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats error management as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, error management must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the error management activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's error management record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the error management aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 42: Deployment Design

Page focus: Define safe interfaces between components. This page provides detailed project-level guidance for **deployment design** within PocketSmart AI.

1. Purpose: This page develops the deployment design aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats deployment design as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, deployment design must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the deployment design activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's deployment design record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the deployment design aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Design — Page 43: Security Architecture

Page focus: Prepare diagrams and design decisions that guide implementation. This page provides detailed project-level guidance for **security architecture** within PocketSmart AI.

1. Purpose: This page develops the security architecture aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats security architecture as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, security architecture must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the security architecture activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's security architecture record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the security architecture aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 44: Work Breakdown Structure

Page focus: Break the project into manageable activities and milestones. This page provides detailed project-level guidance for **work breakdown structure** within PocketSmart AI.

1. Purpose: This page develops the work breakdown structure aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats work breakdown structure as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, work breakdown structure must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the work breakdown structure activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's work breakdown structure record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.

34. Purpose: This page develops the work breakdown structure aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 45: Schedule

Page focus: Estimate effort and identify dependencies. This page provides detailed project-level guidance for **schedule** within PocketSmart AI.

1. Purpose: This page develops the schedule aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats schedule as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, schedule must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the schedule activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's schedule record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the schedule aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 46: Milestones

Page focus: Allocate responsibilities and technical resources. This page provides detailed project-level guidance for **milestones** within PocketSmart AI.

1. Purpose: This page develops the milestones aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats milestones as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, milestones must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the milestones activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's milestones record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the milestones aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 47: Resources

Page focus: Define a practical development sequence. This page provides detailed project-level guidance for **resources** within PocketSmart AI.

1. Purpose: This page develops the resources aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats resources as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, resources must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the resources activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's resources record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the resources aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 48: Roles

Page focus: Prepare contingency plans for common project risks. This page provides detailed project-level guidance for **roles** within PocketSmart AI.

1. Purpose: This page develops the roles aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats roles as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, roles must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the roles activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's roles record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the roles aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 49: Risk Register

Page focus: Break the project into manageable activities and milestones. This page provides detailed project-level guidance for **risk register** within PocketSmart AI.

1. Purpose: This page develops the risk register aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats risk register as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, risk register must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the risk register activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's risk register record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the risk register aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 50: Communication Plan

Page focus: Estimate effort and identify dependencies. This page provides detailed project-level guidance for **communication plan** within PocketSmart AI.

1. Purpose: This page develops the communication plan aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats communication plan as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, communication plan must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the communication plan activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's communication plan record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the communication plan aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 51: Quality Plan

Page focus: Allocate responsibilities and technical resources. This page provides detailed project-level guidance for **quality plan** within PocketSmart AI.

1. Purpose: This page develops the quality plan aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats quality plan as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, quality plan must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the quality plan activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's quality plan record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the quality plan aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 52: Change Management

Page focus: Define a practical development sequence. This page provides detailed project-level guidance for **change management** within PocketSmart AI.

1. Purpose: This page develops the change management aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats change management as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, change management must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the change management activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's change management record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the change management aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 53: Version Control

Page focus: Prepare contingency plans for common project risks. This page provides detailed project-level guidance for **version control** within PocketSmart AI.

1. Purpose: This page develops the version control aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats version control as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, version control must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the version control activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's version control record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the version control aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 54: Sprint Planning

Page focus: Break the project into manageable activities and milestones. This page provides detailed project-level guidance for **sprint planning** within PocketSmart AI.

1. Purpose: This page develops the sprint planning aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats sprint planning as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, sprint planning must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the sprint planning activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's sprint planning record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the sprint planning aspect of PocketSmart AI in a structured, evidence-oriented manner.

Project Planning — Page 55: Deliverables

Page focus: Estimate effort and identify dependencies. This page provides detailed project-level guidance for **deliverables** within PocketSmart AI.

1. Purpose: This page develops the deliverables aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats deliverables as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, deliverables must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the deliverables activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's deliverables record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the deliverables aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 56: Environment Setup

Page focus: Implement the planned architecture in incremental and testable components. This page provides detailed project-level guidance for **environment setup** within PocketSmart AI.

1. Purpose: This page develops the environment setup aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats environment setup as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, environment setup must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the environment setup activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's environment setup record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the environment setup aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 57: Frontend

Page focus: Build secure and usable budgeting workflows. This page provides detailed project-level guidance for **frontend** within PocketSmart AI.

1. Purpose: This page develops the frontend aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats frontend as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, frontend must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the frontend activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's frontend record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the frontend aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 58: Backend

Page focus: Connect the application to persistent financial data. This page provides detailed project-level guidance for **backend** within PocketSmart AI.

1. Purpose: This page develops the backend aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats backend as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, backend must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the backend activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's backend record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the backend aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 59: Database

Page focus: Implement recommendation logic with clear and explainable outputs. This page provides detailed project-level guidance for **database** within PocketSmart AI.

1. Purpose: This page develops the database aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats database as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, database must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the database activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's database record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the database aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 60: Authentication

Page focus: Integrate all modules into a stable working application. This page provides detailed project-level guidance for **authentication** within PocketSmart AI.

1. Purpose: This page develops the authentication aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats authentication as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, authentication must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the authentication activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's authentication record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the authentication aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 61: Expense Entry

Page focus: Implement the planned architecture in incremental and testable components. This page provides detailed project-level guidance for **expense entry** within PocketSmart AI.

1. Purpose: This page develops the expense entry aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats expense entry as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, expense entry must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the expense entry activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's expense entry record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the expense entry aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 62: Income Entry

Page focus: Build secure and usable budgeting workflows. This page provides detailed project-level guidance for **income entry** within PocketSmart AI.

1. Purpose: This page develops the income entry aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats income entry as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, income entry must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the income entry activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's income entry record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the income entry aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 63: Budget Management

Page focus: Connect the application to persistent financial data. This page provides detailed project-level guidance for **budget management** within PocketSmart AI.

1. Purpose: This page develops the budget management aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats budget management as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, budget management must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the budget management activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's budget management record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the budget management aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 64: Categories

Page focus: Implement recommendation logic with clear and explainable outputs. This page provides detailed project-level guidance for **categories** within PocketSmart AI.

1. Purpose: This page develops the categories aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats categories as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, categories must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the categories activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's categories record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the categories aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 65: Dashboard

Page focus: Integrate all modules into a stable working application. This page provides detailed project-level guidance for **dashboard** within PocketSmart AI.

1. Purpose: This page develops the dashboard aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats dashboard as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, dashboard must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the dashboard activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's dashboard record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the dashboard aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 66: Analytics

Page focus: Implement the planned architecture in incremental and testable components. This page provides detailed project-level guidance for **analytics** within PocketSmart AI.

1. Purpose: This page develops the analytics aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats analytics as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, analytics must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the analytics activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's analytics record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the analytics aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 67: Recommendations

Page focus: Build secure and usable budgeting workflows. This page provides detailed project-level guidance for **recommendations** within PocketSmart AI.

1. Purpose: This page develops the recommendations aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats recommendations as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, recommendations must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the recommendations activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's recommendations record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the recommendations aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 68: Alerts

Page focus: Connect the application to persistent financial data. This page provides detailed project-level guidance for **alerts** within PocketSmart AI.

1. Purpose: This page develops the alerts aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats alerts as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, alerts must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the alerts activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's alerts record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the alerts aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 69: Search

Page focus: Implement recommendation logic with clear and explainable outputs. This page provides detailed project-level guidance for **search** within PocketSmart AI.

1. Purpose: This page develops the search aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats search as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, search must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the search activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's search record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the search aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 70: Filters

Page focus: Integrate all modules into a stable working application. This page provides detailed project-level guidance for **filters** within PocketSmart AI.

1. Purpose: This page develops the filters aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats filters as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, filters must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the filters activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's filters record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the filters aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 71: Api Integration

Page focus: Implement the planned architecture in incremental and testable components. This page provides detailed project-level guidance for **API integration** within PocketSmart AI.

1. Purpose: This page develops the API integration aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats API integration as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, API integration must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the API integration activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's API integration record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the API integration aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 72: Exception Handling

Page focus: Build secure and usable budgeting workflows. This page provides detailed project-level guidance for **exception handling** within PocketSmart AI.

1. Purpose: This page develops the exception handling aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats exception handling as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, exception handling must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the exception handling activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's exception handling record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the exception handling aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 73: Logging

Page focus: Connect the application to persistent financial data. This page provides detailed project-level guidance for **logging** within PocketSmart AI.

1. Purpose: This page develops the logging aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats logging as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, logging must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the logging activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's logging record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the logging aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 74: Performance

Page focus: Implement recommendation logic with clear and explainable outputs. This page provides detailed project-level guidance for **performance** within PocketSmart AI.

1. Purpose: This page develops the performance aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats performance as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, performance must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the performance activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's performance record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the performance aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 75: Security

Page focus: Integrate all modules into a stable working application. This page provides detailed project-level guidance for **security** within PocketSmart AI.

1. Purpose: This page develops the security aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats security as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, security must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the security activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's security record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the security aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 76: Deployment

Page focus: Implement the planned architecture in incremental and testable components. This page provides detailed project-level guidance for **deployment** within PocketSmart AI.

1. Purpose: This page develops the deployment aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats deployment as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, deployment must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the deployment activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's deployment record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the deployment aspect of PocketSmart AI in a structured, evidence-oriented manner.

Development — Page 77: Maintenance

Page focus: Build secure and usable budgeting workflows. This page provides detailed project-level guidance for **maintenance** within PocketSmart AI.

1. Purpose: This page develops the maintenance aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats maintenance as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, maintenance must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the maintenance activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's maintenance record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the maintenance aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 78: Test Strategy

Page focus: Verify that each feature behaves according to its requirements. This page provides detailed project-level guidance for **test strategy** within PocketSmart AI.

1. Purpose: This page develops the test strategy aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats test strategy as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, test strategy must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the test strategy activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's test strategy record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the test strategy aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 79: Unit Testing

Page focus: Identify defects before demonstration and release. This page provides detailed project-level guidance for **unit testing** within PocketSmart AI.

1. Purpose: This page develops the unit testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats unit testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, unit testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the unit testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's unit testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the unit testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 80: Integration Testing

Page focus: Validate calculations and recommendation behavior. This page provides detailed project-level guidance for **integration testing** within PocketSmart AI.

1. Purpose: This page develops the integration testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats integration testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, integration testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the integration testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's integration testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the integration testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 81: System Testing

Page focus: Check security, usability, compatibility, and performance. This page provides detailed project-level guidance for **system testing** within PocketSmart AI.

1. Purpose: This page develops the system testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats system testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, system testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the system testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's system testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the system testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 82: Acceptance Testing

Page focus: Produce objective evidence of system quality. This page provides detailed project-level guidance for **acceptance testing** within PocketSmart AI.

1. Purpose: This page develops the acceptance testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats acceptance testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, acceptance testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the acceptance testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's acceptance testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the acceptance testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 83: Regression Testing

Page focus: Verify that each feature behaves according to its requirements. This page provides detailed project-level guidance for **regression testing** within PocketSmart AI.

1. Purpose: This page develops the regression testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats regression testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, regression testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the regression testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's regression testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the regression testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 84: Ui Testing

Page focus: Identify defects before demonstration and release. This page provides detailed project-level guidance for **UI testing** within PocketSmart AI.

1. Purpose: This page develops the UI testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats UI testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, UI testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the UI testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's UI testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the UI testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 85: Api Testing

Page focus: Validate calculations and recommendation behavior. This page provides detailed project-level guidance for **API testing** within PocketSmart AI.

1. Purpose: This page develops the API testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats API testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, API testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the API testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's API testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the API testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 86: Database Testing

Page focus: Check security, usability, compatibility, and performance. This page provides detailed project-level guidance for **database testing** within PocketSmart AI.

1. Purpose: This page develops the database testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats database testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, database testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the database testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's database testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the database testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Testing — Page 87: Security Testing

Page focus: Produce objective evidence of system quality. This page provides detailed project-level guidance for **security testing** within PocketSmart AI.

1. Purpose: This page develops the security testing aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats security testing as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, security testing must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the security testing activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's security testing record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the security testing aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 88: Project Abstract

Page focus: Record the project in a form that another student or evaluator can understand. This page provides detailed project-level guidance for **project abstract** within PocketSmart AI.

1. Purpose: This page develops the project abstract aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats project abstract as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, project abstract must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the project abstract activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's project abstract record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the project abstract aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 89: Introduction

Page focus: Explain design decisions and implementation choices. This page provides detailed project-level guidance for **introduction** within PocketSmart AI.

1. Purpose: This page develops the introduction aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats introduction as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, introduction must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the introduction activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's introduction record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the introduction aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 90: Problem Statement

Page focus: Provide installation and user guidance. This page provides detailed project-level guidance for **problem statement** within PocketSmart AI.

1. Purpose: This page develops the problem statement aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats problem statement as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, problem statement must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the problem statement activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's problem statement record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the problem statement aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 91: Objectives

Page focus: Maintain traceability between requirements, features, and tests. This page provides detailed project-level guidance for **objectives** within PocketSmart AI.

1. Purpose: This page develops the objectives aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats objectives as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, objectives must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the objectives activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's objectives record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the objectives aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 92: Scope

Page focus: Prepare professional academic evidence for evaluation. This page provides detailed project-level guidance for **scope** within PocketSmart AI.

1. Purpose: This page develops the scope aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats scope as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, scope must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the scope activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's scope record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the scope aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 93: Literature Review

Page focus: Record the project in a form that another student or evaluator can understand. This page provides detailed project-level guidance for **literature review** within PocketSmart AI.

1. Purpose: This page develops the literature review aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats literature review as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, literature review must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the literature review activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's literature review record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the literature review aspect of PocketSmart AI in a structured, evidence-oriented manner.

Documentation — Page 94: Methodology

Page focus: Explain design decisions and implementation choices. This page provides detailed project-level guidance for **methodology** within PocketSmart AI.

1. Purpose: This page develops the methodology aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats methodology as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, methodology must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the methodology activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's methodology record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the methodology aspect of PocketSmart AI in a structured, evidence-oriented manner.

Demonstration — Page 95: Demo Preparation

Page focus: Present the working system clearly from login through financial insight generation. This page provides detailed project-level guidance for **demo preparation** within PocketSmart AI.

1. Purpose: This page develops the demo preparation aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats demo preparation as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, demo preparation must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the demo preparation activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's demo preparation record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the demo preparation aspect of PocketSmart AI in a structured, evidence-oriented manner.

Demonstration — Page 96: Opening Explanation

Page focus: Show realistic user workflows rather than isolated screens. This page provides detailed project-level guidance for **opening explanation** within PocketSmart AI.

1. Purpose: This page develops the opening explanation aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats opening explanation as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, opening explanation must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the opening explanation activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's opening explanation record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the opening explanation aspect of PocketSmart AI in a structured, evidence-oriented manner.

Demonstration — Page 97: Login Workflow

Page focus: Explain how ai recommendations are produced. This page provides detailed project-level guidance for **login workflow** within PocketSmart AI.

1. Purpose: This page develops the login workflow aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats login workflow as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, login workflow must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the login workflow activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's login workflow record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the login workflow aspect of PocketSmart AI in a structured, evidence-oriented manner.

Demonstration — Page 98: Profile Setup

Page focus: Demonstrate validation, alerts, analytics, and reporting. This page provides detailed project-level guidance for **profile setup** within PocketSmart AI.

1. Purpose: This page develops the profile setup aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats profile setup as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, profile setup must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the profile setup activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's profile setup record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the profile setup aspect of PocketSmart AI in a structured, evidence-oriented manner.

Demonstration — Page 99: Income Entry

Page focus: Answer evaluator questions with evidence from the implemented project. This page provides detailed project-level guidance for **income entry** within PocketSmart AI.

1. Purpose: This page develops the income entry aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats income entry as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, income entry must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the income entry activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's income entry record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the income entry aspect of PocketSmart AI in a structured, evidence-oriented manner.

Demonstration — Page 100: Expense Entry

Page focus: Present the working system clearly from login through financial insight generation. This page provides detailed project-level guidance for **expense entry** within PocketSmart AI.

1. Purpose: This page develops the expense entry aspect of PocketSmart AI in a structured, evidence-oriented manner.
2. The project team treats expense entry as a controlled engineering activity with a clear input, process, output, and review step.
3. For PocketSmart AI, expense entry must remain aligned with the central goal of helping users understand and manage personal budgets.
4. A useful output of the expense entry activity is a decision that can be documented, reviewed, implemented, and tested. A practical example is the handling of a user's expense entry record from entry through validation and storage.
5. The design should favor clarity, maintainability, security, and user control when decisions affect financial information.
6. User expectations are considered together with technical feasibility, project time, data quality, and academic evaluation needs.
7. The system should avoid unnecessary complexity when a simpler implementation can satisfy the same requirement.
8. Every important decision should have a reason so that later development and testing can trace the decision back to the project objective.
9. Financial calculations should be deterministic and reproducible wherever possible, especially for totals, balances, and budget comparisons.
10. AI-generated recommendations should be treated as assistance and should be presented in understandable language.
11. Sensitive financial information should be handled with appropriate authentication, access control, validation, and secure storage practices. The expected evidence includes a requirement, design note, implementation reference, and corresponding test case.
12. Error conditions should be anticipated because incorrect input can otherwise lead to misleading financial summaries.
13. The team should review assumptions regularly because requirements can change as users and evaluators provide feedback.
14. Usability is important because a budgeting tool is valuable only when users can understand and consistently use it.
15. Performance should be sufficient for normal student-project workloads while leaving a path for future scaling.
16. Data structures should support reliable reporting without creating unnecessary duplication or inconsistency.
17. Interfaces between modules should be explicit so that changes in one component do not unexpectedly break another component.
18. Testing evidence should be collected throughout the project instead of being postponed until the final week. A reviewer should be able to identify the input data, expected behavior, exception path, and final output.
19. Documentation should describe both what was built and why the selected approach was appropriate.
20. Project risks should be recorded with mitigation actions rather than being treated as unexpected events.
21. The team should use version control to preserve history, support collaboration, and make changes recoverable.
22. Security and privacy should be considered from the beginning instead of being added only after implementation.
23. Recommendations should be explainable enough for a user to understand the general reason behind a suggestion.
24. Budget categories should be flexible because different users organize their spending in different ways.
25. The dashboard should emphasize useful information such as income, spending, remaining budget, savings, and important alerts. This approach also makes later maintenance easier because responsibilities are separated and documented.
26. Validation rules should prevent impossible values such as negative expenses or malformed dates when those values are not meaningful.
27. Logs should support troubleshooting without unnecessarily storing sensitive financial details.
28. Future enhancements should be separated from the minimum viable product so that scope remains manageable.
29. Acceptance criteria provide a practical definition of what it means for a feature to be complete.
30. Reviews with stakeholders or evaluators can reveal gaps that are not obvious during individual development.
31. Quality is treated as a combination of correctness, security, usability, maintainability, and reliability.
32. The final outcome should be a coherent system rather than a collection of unrelated screens. The result should be observable in the user interface, application logs, reports, or test evidence as appropriate.
33. Each phase contributes evidence to the next phase, creating a continuous chain from idea to demonstration.
34. Purpose: This page develops the expense entry aspect of PocketSmart AI in a structured, evidence-oriented manner.